

Redução Transitiva de Grafos Direcionados Baseada em Caminhamento em Profundidade

Yrlan Rangel Teixeira ✉ [Pontifícia Universidade Católica de Minas Gerais | yrlan.rangel@sga.pucminas.br]

Hugo Alves Duarte [Pontifícia Universidade Católica de Minas Gerais | hduarte@sga.pucminas.br]

Pedro Henrique Debs Rabelo [Pontifícia Universidade Católica de Minas Gerais | pedrodebs1@gmail.com]

Gabriel Reis Villela [Pontifícia Universidade Católica de Minas Gerais | gabriel.villela@sga.pucminas.br]

Gilson Júnio Pacheco Silva [Pontifícia Universidade Católica de Minas Gerais | gilson.pacheco@sga.pucminas.br]

Daniel Valadares Souza [Pontifícia Universidade Católica de Minas Gerais | danielvsouza25@gmail.com]

Vinicius Figueiredo Ferreira [Pontifícia Universidade Católica de Minas Gerais | vinicius.ferreira.1350368@sga.pucminas.br]

✉ Instituto de Ciências Exatas e Informática, Pontifícia Universidade Católica de Minas Gerais, Av. Dom José Gaspar, 500 - Coração Eucarístico, Belo Horizonte - MG, 30535-901, Brasil.

Received: DD Month YYYY • Accepted: DD Month YYYY • Published: DD Month YYYY

Abstract. A redução transitiva de um grafo direcionado $G = (V, E)$ produz um grafo G' com o menor número de arestas que preserva a atingibilidade de todos os vértices de G . Este trabalho apresenta a implementação, em C++, de um algoritmo de redução transitiva para grafos direcionados acíclicos (DAGs) baseado em caminhamento em profundidade (DFS), com complexidade $O(|V| \cdot (|V| + |E|))$. Descrevemos a representação do grafo adotada (lista de adjacência) e a justificamos, apresentamos um verificador formal de corretude e relatamos experimentos com DAGs aleatórios de até $3,2 \times 10^3$ vértices e 10^6 arestas, nos quais o tempo de execução observado é compatível com a análise assintótica. Discutimos, por fim, por que o conceito de redução transitiva não se transfere diretamente para grafos não direcionados e o que ocorreria com a nossa implementação nesse cenário.

Keywords: Grafos

1 Introdução

Seja $G = (V, E)$ um grafo direcionado. A *atingibilidade* de um vértice $u \in V$ é o conjunto $R_G(u) = \{v \in V \mid \text{existe caminho de } u \text{ a } v \text{ em } G\}$. Em muitas aplicações (compactação de hierarquias de dependências, visualização de grafos, fechados de ordens parciais, redes de citações) interessa o *menor* subgrafo que preserva exatamente essa relação de atingibilidade. Esse subgrafo é a *redução transitiva* de G , definida por Aho, Garey e Ullman Aho *et al.* [1972]: um grafo $G' = (V, E')$, $E' \subseteq E$, tal que (i) $R_{G'}(u) = R_G(u)$ para todo $u \in V$ e (ii) nenhum subgrafo próprio de G' satisfaz (i). Para DAGs a redução transitiva é única: uma aresta (u, v) pertence a G' se, e somente se, v não é alcançável a partir de u por nenhum caminho de comprimento maior ou igual a 2.

Este trabalho implementa a redução transitiva por meio de *caminhamento no grafo*: a decisão de manter ou remover cada aresta é tomada a partir de buscas em profundidade (DFS), sem construir o fecho transitivo completo nem usar multiplicação de matrizes. A Seção 2 detalha o algoritmo e a representação adotada; a Seção 3 apresenta a metodologia experimental e os resultados; a Seção 4 disserta sobre o caso não direcionado; a Seção 5 conclui. As responsabilidades constam da Seção 6.

2 Implementação

O código foi organizado de forma modular, separando interfaces (`include/`) de implementações (`src/`) e dos executáveis (`apps/`): a classe `Grafo` encapsula a repre-

sentação; o módulo `caminhamento` concentra as DFSs; `ReducaoTransitiva`, `Verificador` e `GeradorDag` implementam, respectivamente, o algoritmo, a verificação de corretude e a geração de instâncias; `GrafoIO` isola a E/S. Essa separação permite testar cada componente de forma independente e reutilizar o caminhamento tanto na redução quanto na verificação.

2.1 Representação do grafo

O grafo é representado por **lista de adjacência**: um vetor `adj` de $|V|$ posições em que `adj[u]` é um `std::vector<int>` com os sucessores diretos de u . A escolha é justificada por três fatores. Primeiro, o algoritmo é inteiramente baseado em caminhamento: uma DFS sobre lista de adjacência custa $O(|V| + |E|)$, pois visita apenas as arestas que de fato existem, ao passo que sobre matriz de adjacência custaria $\Theta(|V|^2)$ por busca, mesmo em grafos esparsos; o custo total do algoritmo saltaria, assim, de $O(|V|(|V| + |E|))$ para $\Theta(|V|^3)$. Segundo, a memória é $O(|V| + |E|)$ contra $\Theta(|V|^2)$ da matriz, o que permite tratar grafos grandes e esparsos. Terceiro, a única operação que a matriz ofereceria em $O(1)$ (consultar se uma aresta específica existe) não é necessária: o algoritmo nunca pergunta “existe (u, v) ?”, apenas enumera sucessores. O uso de `std::vector` contíguo ainda favorece a localidade de *cache* durante as travessias.

Entrada: DAG $G = (V, E)$ em lista de adjacência

Saída: redução transitiva $G' = (V, E')$

```

1  $E' \leftarrow \emptyset$ 
2 para cada  $u \in V$  faça
3    $visitado[\cdot] \leftarrow falso$ 
4   para cada  $w \in adj(u)$  faça
5     para cada  $x \in adj(w)$  faça
6       DFS-Marcas( $G, x, visitado$ )
7   para cada  $v \in adj(u)$  faça
8     se  $\neg visitado[v]$  então  $E' \leftarrow E' \cup \{(u, v)\}$ 
9 devolva  $G' = (V, E')$ 
    
```

Algoritmo 1: Redução transitiva por caminhamento

2.2 Algoritmo de redução baseado em caminhamento

A implementação explora a seguinte caracterização, válida para DAGs:

Propriedade. Uma aresta $(u, v) \in E$ é transitiva (redundante) se, e somente se, existe em G um caminho de u até v de comprimento ≥ 2 .

De fato, se tal caminho existe, remover (u, v) não altera $R_G(u)$ (nem de nenhum outro vértice); reciprocamente, se a aresta é removível, a atingibilidade deve ser garantida por um caminho alternativo, necessariamente de comprimento ≥ 2 . Em um DAG esse caminho alternativo nunca pode usar a própria aresta (u, v) “por trás” de um ciclo, o que torna o teste local correto.

O Algoritmo 1 aplica essa propriedade diretamente. Para cada vértice u , o conjunto de vértices alcançáveis por caminhos de comprimento ≥ 2 é exatamente $\bigcup_{w \in adj(u)} \bigcup_{x \in adj(w)} R_G(x)$; ou seja, basta disparar DFSs a partir dos sucessores dos sucessores de u , compartilhando um único vetor de marcação. Ao final, mantém-se em G' somente as arestas (u, v) cujo destino v não foi marcado.

A rotina DFS-Marcas é uma busca em profundidade iterativa (com pilha explícita), evitando estouro da pilha de execução em grafos profundos. Como o vetor *visitado* é compartilhado entre todas as DFSs disparadas para um mesmo u , cada vértice e cada aresta são processados no máximo uma vez por iteração externa, e o custo por vértice u é $O(|V| + |E|)$. O custo total é, portanto,

$$T(|V|, |E|) = O(|V| \cdot (|V| + |E|)),$$

com espaço adicional $O(|V|)$ além do grafo. Antes da redução, o programa executa uma DFS com três cores para certificar que a entrada é acíclica, rejeitando grafos com ciclos; nesse caso a redução transitiva única não é definida sobre subconjuntos de E e o tratamento adequado exigiria condensação em componentes fortemente conexas.

2.3 Verificador de corretude

Para sustentar a correção experimental, implementamos um verificador independente que confere as duas condições da definição: (i) *equivalência de atingibilidade*, comparando $R_G(u)$ e $R_{G'}(u)$ para todo u via DFS em ambos os grafos; e

Table 1. Resultados da redução transitiva em DAGs aleatórios ($G(n, p)$ ordenado, semente fixa).

n	p	$ E $	$ E' $	Remov.	Tempo (ms)
100	0,01	47	47	0%	0,004
100	0,05	223	184	17%	0,047
100	0,20	959	208	78%	0,095
400	0,01	821	782	5%	0,088
400	0,05	3 954	1 324	67%	1,16
400	0,20	15 814	882	94%	3,14
1600	0,01	12 712	6 756	47%	12,4
1600	0,05	63 826	5 715	91%	41,3
1600	0,20	255 386	3 822	98,5%	96,2
3200	0,01	50 938	15 165	70%	93,6
3200	0,05	256 244	11 468	95,5%	233
3200	0,20	1 023 375	7 569	99,3%	658

(ii) *minimalidade*, certificando que nenhuma aresta remanescente de G' é alcançável por caminho alternativo de comprimento ≥ 2 . O verificador foi aplicado a todos os casos de teste, incluindo instâncias clássicas (triângulo transitivo, diamante com atalho) e DAGs aleatórios de diversos tamanhos e densidades, sempre com resultado positivo.

3 Experimentos e resultados

3.1 Metodologia

Os DAGs de entrada foram gerados pelo modelo $G(n, p)$ ordenado: fixada uma permutação topológica implícita $1, \dots, n$, cada par (i, j) com $i < j$ recebe aresta com probabilidade p , o que garante aciclicidade por construção. Variamos $n \in \{100, 200, 400, 800, 1600, 3200\}$ e $p \in \{0,01; 0,05; 0,2\}$, cobrindo regimes esparsos e densos. Os programas foram compilados com g++ -O2 (C++17) e executados em ambiente Linux de 64 bits; os tempos referem-se exclusivamente à rotina de redução (excluindo E/S), medidos com `std::chrono`.

3.2 Resultados

A Tabela 1 resume os resultados. Dois comportamentos merecem destaque. Primeiro, a *taxa de remoção* cresce fortemente com a densidade: para $p = 0,2$, mais de 99% das arestas são transitivas nos grafos maiores ($n \geq 1600$), enquanto para $p = 0,01$ e n pequeno quase nada é removido, o que é coerente com a teoria, já que em DAGs densos a maioria dos pares conectados possui múltiplos caminhos. Segundo, o tempo de execução escala de forma compatível com $O(|V| \cdot (|V| + |E|))$: por exemplo, de $n = 1600$ para $n = 3200$ com $p = 0,2$, $|V|$ dobra e $|E|$ quadruplica, e o tempo cresce $\approx 6,8 \times$ ($96 \text{ ms} \rightarrow 658 \text{ ms}$), próximo do fator 8 previsto pelo termo dominante $|V| \cdot |E|$.

A Figura 1 apresenta o tempo em escala duplamente logarítmica. Um efeito interessante observado é que, para p alto, o tempo cresce *menos* que o pior caso sugere: como quase todas as arestas diretas são alcançadas logo no início das DFSs (que param em vértices já marcados), o caminhamento compartilhado por vértice efetivamente poda grande parte do trabalho.

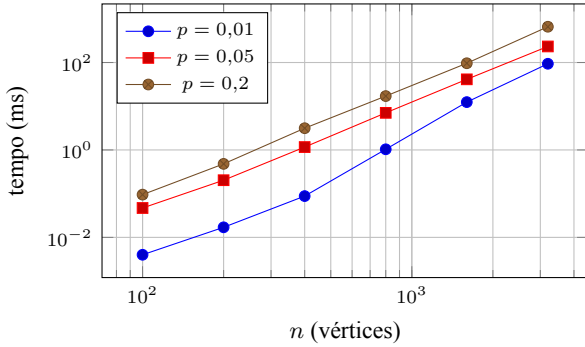


Figure 1. Tempo de execução em função de n (escala duplamente logarítmica). A inclinação aproximadamente constante das curvas é consistente com o crescimento polinomial $O(|V| \cdot (|V| + |E|))$.

4 O caso não direcionado

A implementação **não funcionaria** para grafos não direcionados, por uma razão conceitual anterior a qualquer detalhe de código. Em um grafo não direcionado conexo, todo vértice alcança todos os demais; a atingibilidade de u é simplesmente a componente conexa de u . O subgrafo mínimo que preserva componentes conexas é uma *árvore geradora* de cada componente; ou seja, a “redução transitiva” de um grafo não direcionado degenera no problema de encontrar florestas geradoras, perdendo a unicidade que existe nos DAGs (qualquer árvore geradora serve, e em geral há exponencialmente muitas).

Do ponto de vista operacional, nossa implementação falharia em dois níveis. Primeiro, uma aresta não-direcionada $\{u, v\}$ corresponde ao par direcionado (u, v) e (v, u) , formando um ciclo de comprimento 2: o teste de aciclicidade rejeitaria a entrada imediatamente. Segundo, ainda que o teste fosse suprimido, a Propriedade da Seção 2.2 deixa de valer: ao procurar caminhos de comprimento ≥ 2 de u a v , a DFS encontraria caminhos que usam a própria aresta $\{u, v\}$ no sentido inverso em outro ponto do percurso, concluindo erroneamente que a aresta é redundante. No triângulo $\{a, b\}, \{b, c\}, \{a, c\}$, por exemplo, *toda* aresta possui caminho alternativo de comprimento 2, e o algoritmo removeria as três simultaneamente, desconectando o grafo, quando o correto (como árvore geradora) seria remover exatamente uma. A remoção precisa ser *sequencial e reavaliada* no caso não direcionado (na linha do algoritmo de Kruskal/DFS para árvores geradoras), enquanto no DAG as decisões por aresta são independentes entre si, que é precisamente o que torna o caminhamento direto correto e eficiente.

O caminho correto para grafos direcionados *com ciclos*, por sua vez, seria condensar as componentes fortemente conexas (Tarjan/Kosaraju), reduzir o DAG de condensação e substituir cada componente por um ciclo hamiltoniano interno mínimo, conforme observado por Aho et al. Aho *et al.* [1972].

5 Conclusão

Implementamos em C++ a redução transitiva de DAGs por caminhamento em profundidade, com complexidade $O(|V| \cdot (|V| + |E|))$, representação por lista de adjacência justificada pela natureza do algoritmo, e corretude atestada por um verificador independente de atingibilidade e mini-

malidade. Os experimentos confirmam o comportamento assintótico previsto e evidenciam que, em DAGs densos, a quase totalidade das arestas é redundante, o que reforça o valor prático da operação. Como trabalho futuro, destacamos a extensão a grafos cíclicos via condensação de componentes fortemente conexas e a paralelização do laço externo, trivialmente paralelizável por vértice.

6 Responsabilidades

O trabalho foi integralmente desenvolvido por **Yrlan Rangel Teixeira**: implementação de todos os módulos em C++ (representação do grafo, caminhamentos, algoritmo de redução transitiva, gerador de instâncias e verificador de corretude), execução e análise dos experimentos e redação deste relatório.

References

- Aho, A. V., Garey, M. R., and Ullman, J. D. (1972). The transitive reduction of a directed graph. *SIAM Journal on Computing*, 1(2):131–137. DOI: 10.1137/0201008.